

Universidade do Minho

Algoritmos e Estruturas de Dados

Engenharia de Telecomunicações e Informática

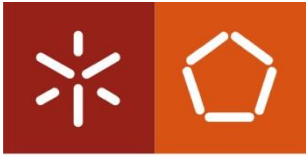
2022/2023

Telmo Adão

Departamento de Sistemas de Informação

Universidade do Minho

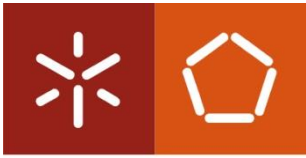
Slides baseados no material didático disponibilizado pelo Professor Luís Magalhães



Universidade do Minho

Sumário

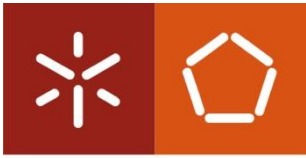
- Análise de algoritmos
 - Efetividade
 - Complexidade e utilização de recursos
- Análise de complexidade dos programas desenvolvidos



Universidade do Minho

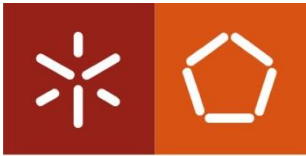
Análise de Algoritmos

- Algoritmo: conjunto claramente especificado de instruções a seguir para resolver um problema
- Análise de algoritmos:
 - Provar que um algoritmo está correto
 - Analisar e determinar os recursos exigidos por um algoritmo (em termos de tempo, espaço, etc.)
 - comparar os recursos exigidos por diferentes algoritmos que resolvem o mesmo problema (um algoritmo mais eficiente exige menos recursos para resolver o mesmo problema)
 - prever o crescimento dos recursos exigidos por um algoritmo à medida que o tamanho dos dados de entrada cresce
- Soluções simples e fáceis de implementar nem sempre são as mais eficientes e até exequíveis!



Análise de Algoritmos

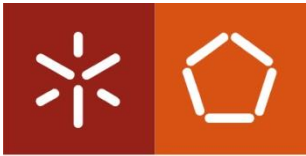
- Tempo de processamento
 - Um algoritmo que realiza uma tarefa em 10 horas é melhor que outro que realiza em 10 dias
- Quantidade de memória necessária
 - Um algoritmo que usa 1MB de memória RAM é melhor que outro que usa 1GB



Análise de Algoritmos

Universidade do Minho

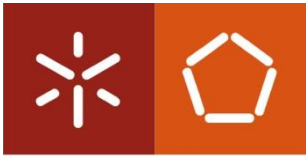
- Geralmente compromisso entre Espaço ocupado/Tempo de execução:
 - Por exemplo para ser mais rápido usam-se estruturas de dados auxiliares
- Estruturas de Dados e Algoritmos são essencialmente úteis para problemas complexos. Para problemas simples qualquer serve:
 - Por exemplo se o espaço de soluções for pequeno usa-se “gerar e testar”
- Na prática (no futuro em projetos que envolvam programação):
 - Se existe feito então usa-se(referindo a fonte ;-))
 - Senão, se existe parecido feito, então adapta-se (referindo a fonte ;-))
 - Senão desenvolve-se de raiz!
 - Adota-se sempre a solução mais simples.



Complexidade Espacial e Temporal

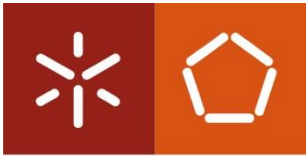
Universidade do Minho

- **Complexidade espacial** de um programa ou algoritmo: espaço de memória que necessita para executar até ao fim
 $S(n)$ - espaço de memória exigido em função do tamanho (n) da entrada
- **Complexidade temporal** de um programa ou algoritmo: tempo que demora a executar (tempo de execução)
 $T(n)$ - tempo de execução em função do tamanho (n) da entrada
- Complexidade $\uparrow$ versus Eficiência $\downarrow$
- Por vezes estima-se a complexidade para:
 - o "melhor caso" (pouco útil)
 - o "pior caso" (mais útil)
 - o "caso médio" (igualmente útil)



Notação de O grande

- Na prática, é difícil (senão impossível) prever com rigor o tempo de execução de um algoritmo ou programa
 - Para obter o tempo a menos de:
 - constantes multiplicativas (normalmente estas constantes são tempos de execução de operações atómicas)
 - parcelas menos significativas para valores grandes de n
 - Identificam-se as operações dominantes (mais frequentes ou muito mais demoradas) e determina-se o número de vezes que são executadas (e não o tempo de cada execução, que seria uma constante multiplicativa)
 - Exprime-se o resultado com a notação de O grande
 - Na prática um contador de operações inserido no programa ou a contagem do tempo são muito úteis para confirmar a complexidade.



Notação de O grande

- Definição:

$$T(n) = O(f(n)) \quad (\text{ler: } T(n) \text{ é de ordem } f(n))$$

se e só se existem constantes positivas c e n_0 tal que $T(n) \leq c \cdot f(n)$ para todo o $n > n_0$

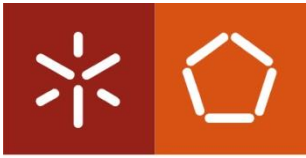
- Exemplos:

$$c_k n^k + c_{k-1} n^{k-1} + \dots + c_0 = O(n^k) \quad (c_i - \text{constantes})$$

$$\log_2 n = O(\log n)$$

(não se indica a base porque mudar de base é multiplicar por constante)

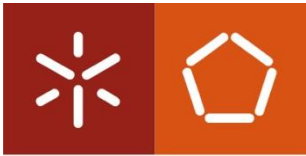
$$4 = O(1) \quad (\text{usa-se } 1 \text{ para ordem constante})$$



Universidade do Minho

Ordens de Crescimento

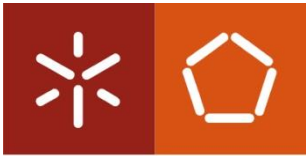
- Classes de Crescimento da complexidade de algoritmos:
 - $O(1)$: constante
 - $O(\log n)$: logarítmico
 - $O(n)$: linear
 - $O(n \log n)$: $n * \log n$
 - $O(n^k)$: polinomial (quadrático, cúbico, etc.)
 - $O(2^n)$: exponencial
 - $O(n!)$: fatorial



Universidade do Minho

Ordens de Crescimento

n	$\log_2 n$	n	$n \times \log_2 n$	n^2	n^3	2^n	$n!$
10	3.3	10	3.3×10^1	10^2	10^3	10^3	3.6×10^6
10^2	6.6	10^2	6.6×10^2	10^4	10^6	1.3×10^{30}	9.3×10^{157}
10^3	10	10^3	1.0×10^4	10^6	10^9	?	?
10^4	13	10^4	1.3×10^4	10^8	10^{12}	?	?
10^5	17	10^5	1.7×10^6	10^{10}	10^{15}	?	?
10^6	20	10^6	2.0×10^7	10^{12}	10^{18}	?	?



Universidade do Minho

Ordens mais comuns

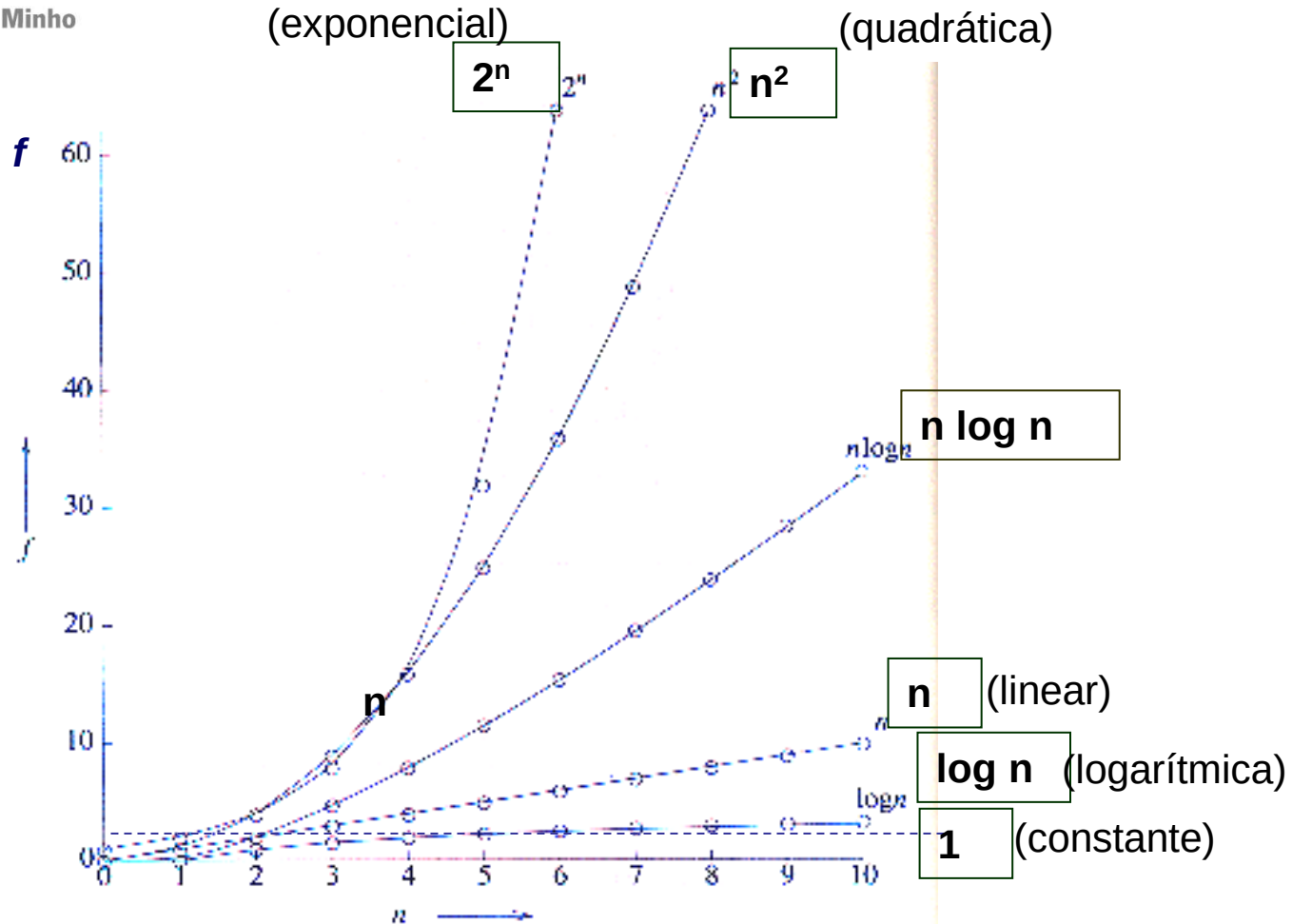
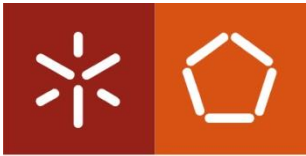


Figure 2.24 Plot of various functions

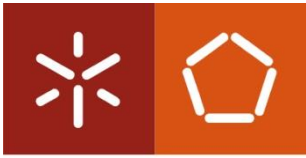
Fonte: Sahni, "Data Structures, Algorithms and Applications in C++"



Termo Dominante

Universidade do Minho

- Suponha que se usa N^3 para estimar $N^3 + 350N^2 + N$
- Para $N = 10000$
 - Valor real = 1 003 500 010 000
 - Valor estimado = 1 000 000 000 000
 - Erro = 0.35% (não é significativo)
- Para valores elevados de N
 - o termo dominante é indicativo do comportamento do algoritmo
- Para valores pequenos de N
 - o termo dominante não é necessariamente indicativo do comportamento, mas geralmente, programas executam tão rapidamente que não importa



Universidade do Minho

Metodologia para determinar a complexidade

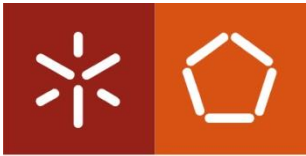
- Considere-se o seguinte algoritmo:

Para $i \leftarrow 1$ até n passo 1

 instruções

Fim_para

- A contabilização do numero de instruções é simples:
 - n iterações e, em cada uma, são executadas um número constante de instruções: $O(n)$



Universidade do Minho

Metodologia para determinar a complexidade

- Considere-se o seguinte código:

Para $i \leftarrow 1$ até n passo 1

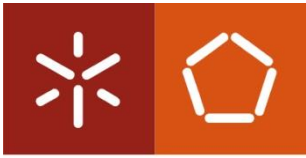
Para $j \leftarrow 1$ até n passo 1

 instruções

Fim_para

Fim_para

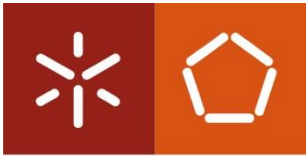
- A contabilização do numero de instruções é ainda simples:
 - o ciclo interno (Para j) é $O(n)$ e é executado n vezes: $O(n^2)$



Universidade do Minho

Exercícios tipo

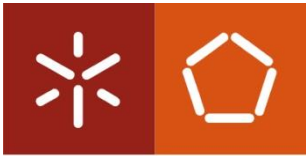
```
int soma_acumulada(int n)
{
    int i;
    int acumulador = 0;
    for(i = 0; i < n; i++)
        { acumulador += i; }
    return acumulador;
}
```



Exercícios tipo

```
- int soma_acumulada(int n) {  
-     int i;  
1     int acumulador = 0;  
n     for(i = 0; i < n; i++) {  
n         acumulador += i;  
-     }  
1     return acumulador;  
- }
```

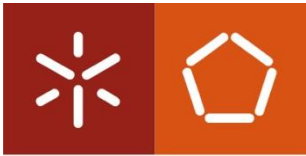
R: $O(n)$



Exercícios tipo

```
- void exemplo(int n)
- {
-     int i, j;
1     int a = 0;
n     for(i = 0; i < n; i++)
n(n+1)/2         for(j = n; j > i; j--)
n(n+1)/2             a += i + j;
1     exemplo1(n);
- }
```

R: $O(n^2)$



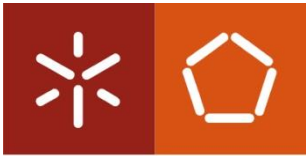
Universidade do Minho

Exercícios tipo

```
int exemplo2(int n)
{
    int i;
    int a = 0;
    for(i = 0; i < n; i+=2)
        { a += i; }
    return a;
}
```

O ciclo for é executado $n/2$ vezes

R: **$O(n)$**



Universidade do Minho

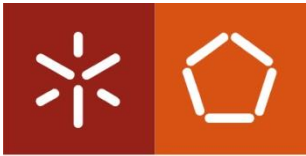
Exercícios tipo

```
int exemplo3(int n)
{
    int i;
    int a = 0;
    for(i = 1; i < n; i*=2)
        { a += i; }
    return a;
}
```

O ciclo termina quando $i=n$

Como $i=2^k$ então $2^k=n$, sendo k o número de iterações,
logo $k=\log_2 n$

R: **$O(\log n)$**



Universidade do Minho

Exercícios tipo

```
int exemplo4(int n)
{
    if (n==0) return n;
    else return n+exemplo4(n-1);
}
```

O return é invocado n vezes

R: **$O(n)$**